

Reflective Journal: IIoT Protocols Project

Introduction

For this project, I simulated an Industrial Internet of Things (IIoT) sensor network using three communication protocols: MQTT, CoAP, and OPC UA. The goal was to generate sensor data, transmit it using each protocol, and visualize the results to better understand how these technologies behave in practice. My personal expectation was to move beyond theory and actually see how industrial protocols are configured, coded, and debugged in a real development environment.

Personal Contributions

My main responsibilities were setting up the project environment on Windows, creating and editing the Python scripts for the three sensor simulations, and getting the MQTT data visualization working. I helped create the `mqtt_sensor_simulation.py`, `coap_sensor_simulation.py`, `opcua_sensor_simulation.py`, and `data_visualization.py` files, and I tested them using a local Mosquitto broker and virtual environment. I also captured screenshots of the real-time plots and contributed to the written documentation, including the README and parts of the comparison and reflection on the three protocols.

Learning Outcomes

From MQTT, I learned how a lightweight publish-subscribe model works and how easy it is to continuously send telemetry data from a simulated sensor to a broker and then to a subscriber. CoAP showed me a different style of communication, more like a small, efficient version of HTTP that uses request/response messages over UDP. With OPC UA, I saw how industrial systems use a richer data model and server architecture, where variables like Temperature and Humidity are exposed as nodes that clients can read from or subscribe to. Overall, the hands-on work helped me understand not just definitions, but how each protocol is actually configured, coded, and used in an IIoT scenario.

Challenges and Solutions

One of the biggest challenges was getting everything to work correctly on Windows, especially setting up the Python virtual environment and installing the required libraries. I also had to deal with the Mosquitto MQTT broker service and make sure it was running on the correct port before the MQTT scripts could connect. Another challenge was understanding how to organize the project files, run multiple terminal windows at the same time, and verify that messages were really being sent and received. I overcame these problems by carefully following step-by-step instructions, checking error messages in the terminal, and testing each script separately before running them together. This process taught me to troubleshoot systematically instead of giving up when something did not work the first time.

Future Applications

The experience from this project will be useful for any future work that involves IoT, industrial automation, or cloud integration. Knowing the differences between MQTT, CoAP, and OPC UA helps me choose the right protocol depending on the devices, network conditions, and data requirements in a system. In a professional setting, I could apply this knowledge to design architectures where MQTT handles telemetry to the cloud, OPC UA connects to factory equipment, and CoAP is used for constrained edge devices. If I had more time, I would extend the project by adding more sensor types, improving the visualizations, and collecting performance metrics to compare latency and reliability between the three protocols in a more quantitative way.